

Next JS

1. React

- ↳ components based architecture
- ↳ State managements
- ↳ UI rendering
- ↳ you manage routing / backend separately

vs

2. Next JS

- ↳ framework built on top of react
- ↳ Gives routing automatically
- ↳ can handle frontend + backend together
- ↳ Supports SSR for faster loading + SEO

```
npm create next-app@latest
```

or

```
npx create-next-app@latest
```

Folder structure :

→ app /

- main application folder (App router in Next.js)
- contains pages / layouts
- controls routing
- UI starts here

→ app /

- page.tsx → Home page
- layout.tsx → Shared layout / navbar.
- global.css → global styles. (convention to follow)

- public /

- Image / static files

- package.json → deps / scripts

- next.config.js → setting of nextjs

- tsconfig.json → Typescript settings

- eslint.config.mjs → code quality rules

- postcss.config.mjs → CSS processing config

- bun.lock → Exact pkg versions

File base routing

- file + folder = URLs

about ---> /about

contact ---> /contact

→ you don't manually configure routes

→ Next.js automatically creates routes from filename inside app/

e.g.,

app/

```
├── page.tsx
├── about/
│   └── page.tsx
└── contact/
    └── page.tsx
```

points to root → /

Nexus is watching you

lol

```
import React from 'React'
```

```
const AgentPage = () => {
```

```
  return (
```

```
    <div> Hello agents </div>
```

```
  )
```

```
}
```

```
export default AgentPage
```

← app/agents/page.tsx

just simple route
that shows
"Hello agents"

layout.tsx → wrapper of all pages

main idea → {children} , that's a placeholder

so like, file has a lot of code and

<body> ---- </body> , in between body , we

render children.. , → current page content.

so, <body>

<div> header </div>

{children}

→ your current page
content

<div> footer </div>

</body>

Dynamic routes

e.g., app /
 └──> blog /
 └──> [id] /
 └── page.tsx

• What's id?

/blog / 1

/blog / 2

/blog / hello

all of em use same page file

• How to access ?

```
export default function Page ({  
  params,
```

```
}) {
```

```
  params: {id: string};
```

```
}) {
```

```
  return <h1> {params.id} </h1>;
```

```
}
```

legacy syntax

/ old way

→ In newer version of next.js

'params' is a promise

→ so use `async / await`

• e.g., blog/html, blog/css, blog/js

```
const BlogPage = async ({
  params,
}) => {
  const { slug } = await params;
  return <h1> Blog on { slug } </h1>;
}
```

```
export default BlogPage
```

blog / css



Blog on CSS.

multiple dynamic routes segments

App/

--- blogs/

--- [category]

--- [slug]

--- page.tsx

```
const BlogPage = async ({
  params,
}) => {
  const { category, slug } = await params;
  return <div> category: { category } <br> blog: { slug } </div>;
}
```

```
export default BlogPage;
```

```
const { category, slug } = await params;
return (<div> category: { category } <br> blog: { slug } </div>);
```

- Catch all routes : → when you don't know how many segments.

→ and how deeply nested route is.

e.g. url: /blog/frontend/react/hooks

example : app/
 !-- blog /
 !--- [... slug] /
 !-- page.tsx

array

```
const BlogPage = async ({
  params,
}) => {
  params: Promise < { slug: string[] } >;
} => {
  const { slug } = await params;
  return <h1> { slug.join("/") } </h1>;
};
export class BlogPage.
```

- Optional Catch all routes: Same as catch-all, but base route also works.

• app / blog / [[... slug]] / page.tsx

```
const BlogPage = async ({
  params,
}) => {
  params: Promise < {
    slug? string[];
  } >;
} => {
  const { slug } = await params;
```

```

return (
  <div>
    {slug}
    ? 'Blog on ${slug.join("/")}'
    : "All blogs"
  </div>
);
};
export default BlogPage;

```

/blogs → All blogs

/blog/react/hooks → blog on react/hooks

So, thats all about ... it

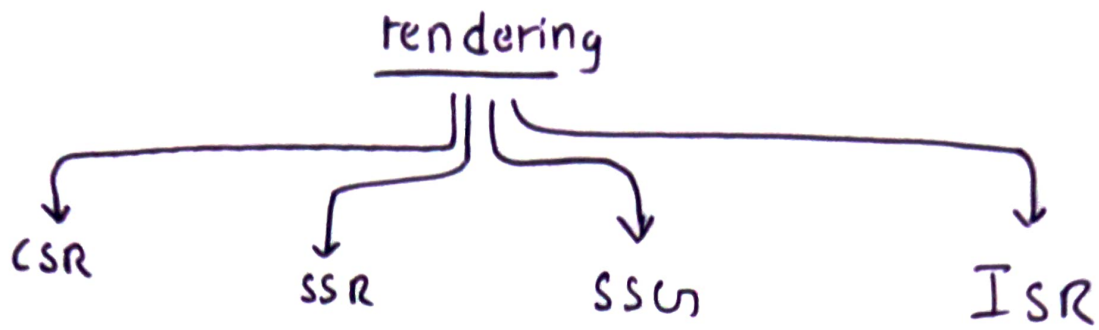
Route groups → ()

```

app/
├── (marketing) /
│   ├── about /
│   │   └── page.tsx
│   └── contact /
│       └── page.tsx

```

} Round bracket
grouping the
routes



CSR: Client side rendering

- Server sends mostly empty HTML + JS
- Browser fetches data and renders UI

SSR: Server side rendering

Server renders HTML on every request (fresh data each time)

SSG: Static site generation

HTML is generated at build time (one time).
Served super fast via CDN

ISR: Incremental static generation

- generate static page now but refresh it later automatically
- Static page + Occasional background updates.

TTFB → Time to First byte.

basically time browser has to wait .. before getting the first piece of data from server.

- user opens website
- browser sends req to server
- next.js runs server side code
- server prepares html
- server start sending response

this complete time b/w step 1 to step 5 is basically 'ttfb'.

Link component

⇒ Import React from 'react'

import Link from "next/link";

C.g, <Link href="/about">

 About
 </Link>

⇒ Important props

- href: Where to go c.g href="/about"

- prefetch: preloads page before clicking
 prefetch = {false}

replace → replaces browser history

Scroll → controls scroll behaviour

Image component:

```
import Image from "next/image";
```

```
B.g, <Image  
      src="/cat.jpg" *  
      alt="cat"  
      width={300}  
      height={200}  
    />
```

- props like priority → loads image fully
- for some image configure host name if fetching from url (SSR does on error)

"use client" : By default pages are server components

If component needs

- useState
- useEffect
- click handlers --

add

"use client"; on top

B.g,

```
"use client";
```

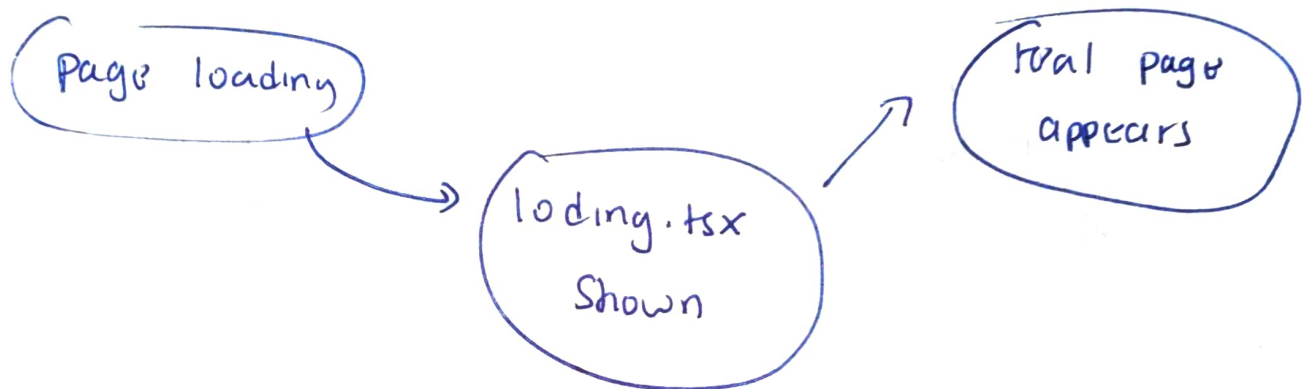
```
import { useState } from "react";
```

→ loading.tsx

Special file for loading UI.

e.g, `app/products/loading.tsx`

e.g, export default function Loading () {
 return <div> Loading </div>;
}



⇒ `error.tsx` : handles page crash automatically

"use client";

export default function Error ({

error,
 reset,
}) {

return {

<div>

<p> {error.message} </p>

<button onClick={ () => reset() } >

Try Again

</button>

</div>

};

error → contains actual error
reset → retries page rendering

} imp things

4 Font optimization : Next.js optimizes font automatically

```
import { Geist } from "next/font/google";
```

```
const geist = Geist({  
  subsets: ["latin"]  
});
```

Apply → `<body className={geist.className}>`